Tugas Besar 2
Berry-Manuka

IF2211 Strategi Algoritma

Pemanfaatan Algoritma BFS dan DFS dalam Mekanisme Penelusuran CSS pada Pohon Document Object Model

Semester II Tahun 2025/2026

Disusun oleh:

Berry-Manuka

Agatha Tatianingseto	(13524008)
Stefani Angeline Oroh	(13524064)
Athilla Zaidan Zidna Fann	(13524068)

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1 Pendahuluan **3**
1.1 Kisah . . . 3
1.2 Latar Belakang . . . 4
1.3 Deskripsi Tugas . . . 4
1.3.1 Masukan . . . 5
1.3.2 Keluaran . . . 5

2 Dasar Teori **6**
2.1 Document Object Model (DOM) . . . 6
2.2 HTML dan HTML Parsing . . . 7
2.2.1 Struktur Dasar HTML . . . 7
2.2.2 HTML Parsing . . . 7
2.3 CSS Selector . . . 8
2.3.1 Selector Dasar . . . 8
2.3.2 Combinator . . . 9
2.3.3 Selector Chain dan Matching . . . 9
2.4 Breadth-First Search (BFS) . . . 9
2.4.1 Karakteristik BFS . . . 9
2.4.2 Pseudocode BFS . . . 10
2.4.3 Analisis Kompleksitas . . . 10
2.5 Depth-First Search (DFS) . . . 11
2.5.1 Karakteristik DFS . . . 11
2.5.2 Pseudocode DFS . . . 11
2.5.3 Analisis Kompleksitas . . . 11
2.6 Perbandingan BFS dan DFS . . . 11
2.6.1 Pemilihan Algoritma yang Tepat . . . 13
2.7 Web Scraping . . . 14
2.7.1 HTTP Client dan Request Handling . . . 14
2.7.2 Validasi dan Error Handling . . . 14
2.7.3 Security Considerations . . . 14
2.8 Lowest Common Ancestor dan Binary Lifting . . . 15
2.8.1 Definisi LCA . . . 15
2.8.2 Pendekatan Naive . . . 15
2.8.3 Binary Lifting . . . 15
2.9 Konkurensi dan Goroutine . . . 16
2.9.1 Goroutine . . . 16

2.9.2 Sinkronisasi dengan WaitGroup . . . 17
2.9.3 Thread Safety dengan Mutex . . . 17
2.9.4 Parallel BFS dan DFS . . . 17
2.10 Containerization dan Docker . . . 17
2.10.1 Konsep Container . . . 18
2.10.2 Docker . . . 18
2.10.3 Dockerfile dan Multi-Stage Builds . . . 18
2.10.4 Docker Compose . . . 19

3 Aplikasi BFS dan DFS **20**
3.1 Representasi DOM Tree . . . 20
3.2 HTML Parsing . . . 20
3.3 Implementasi CSS Selector . . . 20
3.4 Implementasi Algoritma BFS dan DFS . . . 21
3.4.1 Implementasi Algoritma BFS . . . 21
3.4.2 Implementasi Algoritma DFS . . . 23
3.4.3 Perbandingan BFS dan DFS . . . 25
3.4.4 Analisis Kompleksitas . . . 26
3.5 Implementasi Binary Lifting untuk LCA pada DOM . . . 26

4 Implementasi **33**
4.1 Gambaran Umum Sistem . . . 33
4.2 Arsitektur Sistem . . . 33

5 Penutup **35**
5.1 Kesimpulan . . . 35
5.2 Saran . . . 35

6 Lampiran **36**

Bab 1

Pendahuluan

1.1 Kisah

Setiap kali seseorang membuka sebuah halaman web di peramban (*browser*), terjadi serangkaian proses yang kompleks di balik layar. Peramban mengunduh dokumen *HyperText Markup Language* (HTML) dari peladen web, kemudian mengurai dokumen tersebut menjadi sebuah struktur pohon yang disebut *Document Object Model* (DOM). Struktur pohon inilah yang menjadi fondasi bagi seluruh proses selanjutnya — mulai dari penerapan gaya (*styling*) melalui *Cascading Style Sheets* (CSS), manipulasi konten secara dinamis melalui JavaScript, hingga proses *rendering* visual di layar pengguna.

Salah satu mekanisme fundamental dalam pemrosesan halaman web adalah **pencocokan CSS Selector**. Ketika seorang pengembang web menuliskan aturan CSS seperti `div > p.highlight`, peramban harus menelusuri seluruh pohon DOM untuk menemukan elemen-elemen yang memenuhi kriteria tersebut. Proses penelusuran ini pada dasarnya adalah sebuah *tree traversal* — mengunjungi node-node dalam pohon secara sistematis hingga seluruh elemen yang cocok ditemukan.

Dua algoritma traversal yang paling fundamental adalah *Breadth-First Search* (BFS) dan *Depth-First Search* (DFS). Kedua algoritma ini memiliki strategi penelusuran yang berbeda: BFS menjelajahi pohon secara melebar (*level-by-level*), sementara DFS menjelajahi pohon secara mendalam (*sedalam mungkin baru mundur*). Pemilihan algoritma traversal yang tepat dapat memengaruhi efisiensi dan urutan penemuan elemen yang sesuai dengan CSS Selector yang diberikan.

Pada Tugas Besar 2 mata kuliah IF2211 Strategi Algoritma ini, mahasiswa diminta untuk membangun sebuah aplikasi web yang mengimplementasikan proses penelusuran CSS Selector pada pohon DOM menggunakan algoritma BFS dan DFS. Program yang dikembangkan diberi nama **Berry-Manuka**: aplikasi web interaktif yang mampu mengambil dokumen HTML dari suatu URL, mem-parse-nya menjadi pohon DOM, menelusuri pohon tersebut menggunakan BFS maupun DFS untuk mencari elemen yang cocok dengan CSS Selector tertentu, serta memvisualisasikan seluruh proses penelusuran secara interaktif.

1.2 Latar Belakang

Document Object Model (DOM) adalah antarmuka pemrograman yang merepresentasikan struktur dokumen HTML dalam bentuk pohon hierarkis. Setiap elemen HTML dalam dokumen direpresentasikan sebagai sebuah *node* dalam pohon, dengan hubungan *parent-child-sibling* yang mencerminkan struktur bersarang (*nesting*) dari tag-tag HTML [1]. Representasi pohon ini bersifat universal dan menjadi dasar bagi hampir semua operasi yang dilakukan peramban terhadap halaman web.

Pencocokan CSS Selector merupakan operasi yang sangat sering dilakukan oleh peramban. Setiap kali sebuah halaman web dimuat, peramban harus mencocokkan setiap aturan CSS dalam *stylesheet* dengan elemen-elemen yang sesuai di pohon DOM. Proses ini memerlukan penelusuran pohon DOM secara efisien, mengingat kompleksitas halaman web modern yang dapat memiliki ribuan hingga puluhan ribu node [2].

Algoritma *Breadth-First Search* (BFS) dan *Depth-First Search* (DFS) merupakan dua strategi penelusuran graf dan pohon yang paling dasar dan penting dalam ilmu komputer [3]. Kedua algoritma ini memiliki karakteristik yang berbeda dalam hal urutan kunjungan node, penggunaan memori, dan kesesuaian dengan jenis pencarian tertentu. Pemahaman mendalam mengenai perbedaan keduanya sangat penting dalam konteks penelusuran pohon DOM.

Visualisasi interaktif dari proses penelusuran pohon DOM dapat membantu pengguna memahami cara kerja algoritma BFS dan DFS secara lebih intuitif. Daripada hanya membaca pseudocode atau analisis teoritis, pengguna dapat mengamati secara langsung bagaimana setiap algoritma mengunjungi node-node dalam pohon, mengetahui node mana yang diperiksa terlebih dahulu, dan melihat jalur yang ditempuh hingga elemen yang dicari ditemukan.

Untuk meningkatkan performa penelusuran pada pohon DOM yang berukuran besar, diterapkan mekanisme konkurensi menggunakan *goroutine* yang disediakan oleh bahasa pemrograman Go. Dengan pemrosesan paralel, beberapa node dapat diperiksa secara bersamaan sehingga waktu penelusuran dapat dikurangi secara signifikan.

Selain itu, diimplementasikan pula algoritma *Lowest Common Ancestor* (LCA) dengan metode *Binary Lifting* untuk menentukan ancestor bersama terdekat dari dua buah node dalam pohon DOM. Algoritma ini memiliki kompleksitas query sebesar $O(\log N)$, jauh lebih efisien dibandingkan pendekatan naif yang memiliki kompleksitas $O(N)$ per query [4].

1.3 Deskripsi Tugas

Program Berry-Manuka merupakan aplikasi web yang menerima masukan berupa dokumen HTML — baik melalui *URL scraping* maupun input manual — kemudian mem-parse dokumen tersebut menjadi pohon DOM dan menyediakan fasilitas penelusuran pohon menggunakan algoritma BFS atau DFS berdasarkan CSS Selector yang diberikan pengguna. Hasil penelusuran divisualisasikan secara interaktif lengkap dengan animasi proses penelusuran, statistik performa, dan traversal log.

Program dikembangkan menggunakan bahasa pemrograman Go untuk *backend* dan framework Next.js untuk *frontend*, serta dikemas dalam kontainer Docker untuk memudahkan proses deployment. Program ini dikerjakan secara berkelompok oleh tiga orang mahasiswa.

1.3.1 Masukan

Program menerima beberapa parameter masukan melalui antarmuka web:

1. **Sumber dokumen HTML**: dapat berupa URL website yang akan di-*scrape* atau teks HTML yang dimasukkan secara manual oleh pengguna.
2. **Pilihan algoritma traversal**: pilihan antara *Breadth-First Search* (BFS) atau *Depth-First Search* (DFS) sebagai metode penelusuran pohon DOM.
3. **CSS Selector**: selector yang digunakan untuk mencari elemen yang sesuai dalam pohon DOM. Selector yang didukung meliputi *tag selector* (misalnya `p`), *class selector* (misalnya `.highlight`), *ID selector* (misalnya `#header`), dan *universal selector* (`*`), serta *combinator* berupa child (`>`), descendant (spasi), adjacent sibling (`+`), dan general sibling (`~`).
4. **Jumlah hasil**: banyaknya kemunculan elemen yang ingin ditampilkan, baik berupa N hasil pertama (*top-N*) maupun seluruh kemunculan.

1.3.2 Keluaran

Program menghasilkan beberapa bentuk keluaran melalui antarmuka web:

1. **Visualisasi pohon DOM**: representasi visual dari struktur pohon DOM hasil *parsing*, lengkap dengan keterangan kedalaman maksimum (*maximum depth*) dari pohon tersebut.
2. **Highlight jalur traversal**: penandaan visual pada node-node yang dikunjungi selama proses penelusuran, sehingga pengguna dapat mengamati urutan dan jalur yang ditempuh oleh algoritma.
3. **Animasi penelusuran**: animasi langkah demi langkah dari proses penelusuran pohon DOM yang menunjukkan node mana yang sedang diperiksa pada setiap tahapan.
4. **Statistik performa**: informasi mengenai waktu pencarian (dalam milidetik) dan banyaknya node yang dikunjungi selama proses penelusuran.
5. **Traversal log**: catatan berurutan yang memuat informasi setiap tahapan penelusuran, termasuk node yang dikunjungi dan status pencocokan terhadap CSS Selector.
6. **Lowest Common Ancestor**: hasil pencarian ancestor bersama terdekat dari dua buah node yang dipilih oleh pengguna dalam pohon DOM, beserta jalur dari masing-masing node menuju ancestor tersebut.

Bab 2

Dasar Teori

Bagian ini membahas konsep-konsep teoritis yang menjadi landasan pengembangan program Berry-Manuka. Pemahaman terhadap konsep-konsep berikut diperlukan untuk memahami cara kerja aplikasi penelusuran pohon DOM menggunakan algoritma *Breadth-First Search* (BFS) dan *Depth-First Search* (DFS) yang diimplementasikan.

2.1 Document Object Model (DOM)

Document Object Model (DOM) adalah antarmuka pemrograman yang merepresentasikan struktur dokumen HTML dalam bentuk pohon hierarkis [1]. DOM menyediakan representasi terstruktur dari dokumen, memungkinkan manipulasi konten, struktur, dan gaya dokumen melalui bahasa pemrograman seperti JavaScript.

Dalam representasi pohon DOM, setiap elemen HTML dalam dokumen direpresentasikan sebagai sebuah *node* dalam pohon. Terdapat beberapa tipe node yang penting:

- **Element node**: merepresentasikan tag HTML seperti `<div>`, `<p>`, atau `<h1>`. Setiap element node dapat memiliki atribut dan children.
- **Text node**: merepresentasikan teks yang terdapat di dalam elemen HTML. Text node selalu menjadi *leaf node* dalam pohon DOM.
- **Attribute node**: merepresentasikan atribut dari sebuah elemen HTML seperti `class`, `id`, atau `href`.

Struktur pohon DOM memiliki hubungan hierarkis yang mencerminkan struktur bersarang dari tag-tag HTML:

- **Root node**: *node* paling atas dalam hierarki, biasanya elemen `<html>`, yang merepresentasikan keseluruhan dokumen.
- **Parent-child relationship**: setiap node (kecuali root) memiliki tepat satu *parent node*, dan setiap node dapat memiliki nol atau lebih *child nodes*. Jika node A secara langsung mengandung node B di dalamnya, maka A adalah parent dari B dan B adalah child dari A.

- **Sibling relationship**: node-node yang memiliki parent yang sama disebut *siblings*. Siblings memiliki *previous sibling* dan *next sibling* yang menunjukkan urutan kemunculan mereka dalam dokumen.

Setiap node dalam pohon DOM memiliki kedalaman (*depth*) yang menunjukkan jarak dari root node. Root node berada pada kedalaman 0, children langsungnya pada kedalaman 1, dan seterusnya. Kedalaman maksimum (*maximum depth*) dari pohon menunjukkan tinggi pohon tersebut.

Representasi pohon DOM bersifat universal dan menjadi dasar bagi hampir semua operasi yang dilakukan peramban terhadap halaman web, termasuk penerapan gaya CSS, manipulasi konten dinamis, dan pemrosesan event [5].

2.2 HTML dan HTML Parsing

HyperText Markup Language (HTML) adalah bahasa markup standar yang digunakan untuk membuat dan menyusun struktur halaman web [6]. HTML menggunakan sistem *tag* untuk menandai berbagai elemen dalam dokumen seperti teks, gambar, tautan, dan elemen lainnya.

2.2.1 Struktur Dasar HTML

Tag HTML ditulis dengan menggunakan kurung siku sudut (`< >`). Sebagian besar tag memiliki tag pembuka (misalnya `<p>`) dan tag penutup (misalnya `</p>`) yang mengapit konten di dalamnya. Terdapat juga tag yang bersifat *self-closing* atau *void elements* yang tidak memerlukan tag penutup, seperti `<br/>`, `<img/>`, `<input/>`, `<hr/>`, `<meta/>`, dan `<link/>`.

Setiap elemen HTML dapat memiliki atribut yang memberikan informasi tambahan atau mengubah perilaku elemen tersebut. Atribut ditulis di dalam tag pembuka dengan format `nama="nilai"`. Atribut umum yang sering digunakan antara lain:

- `class`: menentukan satu atau lebih kelas CSS untuk elemen
- `id`: memberikan identifier unik untuk elemen dalam dokumen
- `href`: menentukan URL tujuan untuk elemen tautan (`<a>`)
- `src`: menentukan sumber file untuk elemen media (`<img>`, `<script>`)

2.2.2 HTML Parsing

HTML Parsing adalah proses membaca dokumen HTML dan mengubahnya menjadi struktur pohon DOM [7]. Parser memproses dokumen secara berurutan dari awal hingga akhir, mengenali tag-tag HTML dan menyusun hierarki node-node DOM.

Proses parsing HTML melibatkan beberapa langkah utama:

1. **Tokenization**: membaca karakter demi karakter dari dokumen HTML dan mengubahnya menjadi *token* seperti *start tag*, *end tag*, *attribute*, dan *text*.

2. **Tree Construction**: membangun pohon DOM berdasarkan token-token yang dihasilkan. Proses ini menggunakan *stack* untuk melacak node-node yang sedang aktif:
   - Saat menemukan *StartTagToken*, sebuah node baru dibuat, ditambahkan sebagai child dari node di atas stack, kemudian node baru tersebut di-*push* ke stack.
   - Saat menemukan *EndTagToken*, stack di-*pop* untuk kembali ke parent node.
   - Saat menemukan *SelfClosingTagToken*, node dibuat dan ditambahkan sebagai child, tetapi tidak di-*push* ke stack.
   - *TextToken* ditambahkan sebagai *innerText* dari node yang sedang berada di atas stack.

Dalam implementasi program Berry-Manuka, parsing HTML dilakukan menggunakan pustaka `golang.org/x/net/html` yang menyediakan *tokenizer* HTML standar untuk bahasa pemrograman Go. Pustaka ini mampu menangani berbagai kasus edge seperti tag yang tidak tertutup dengan benar, tag yang bersarang secara tidak valid, dan karakter khusus dalam dokumen HTML.

2.3 CSS Selector

Cascading Style Sheets (CSS) Selector adalah pola yang digunakan untuk memilih elemen-elemen tertentu dalam pohon DOM agar dapat diberi aturan gaya [2]. Selector memungkinkan pengembang web untuk menargetkan satu atau lebih elemen berdasarkan tipe tag, atribut class, atribut id, atau hubungan struktural dengan elemen lain.

2.3.1 Selector Dasar

Program Berry-Manuka mendukung empat tipe selector dasar:

- **Tag Selector**: memilih semua elemen dengan nama tag tertentu. Sintaks: `tagname`. Contoh: `p` memilih semua elemen `<p>`.
- **Class Selector**: memilih elemen yang memiliki atribut `class` dengan nilai tertentu. Sintaks: `.classname`. Contoh: `.box` memilih elemen dengan `class="box"`.
- **ID Selector**: memilih elemen yang memiliki atribut `id` dengan nilai tertentu. ID bersifat unik dalam satu dokumen. Sintaks: `#idname`. Contoh: `#header` memilih elemen dengan `id="header"`.
- **Universal Selector**: memilih semua elemen dalam dokumen. Sintaks: `*`.

2.3.2 Combinator

Combinator adalah karakter khusus yang digunakan untuk mengekspresikan hubungan antara dua selector, memungkinkan pemilihan elemen berdasarkan posisi relatif mereka dalam pohon DOM. Terdapat empat tipe combinator:

- **Child Combinator** (>): memilih elemen yang merupakan *direct child* dari elemen induk tertentu. Contoh: `div > p` memilih elemen `<p>` yang secara langsung berada di dalam `<div>`.
- **Descendant Combinator** (spasi): memilih elemen yang merupakan keturunan (bukan harus direct child) dari elemen tertentu. Contoh: `div p` memilih semua `<p>` yang berada di dalam `<div>` pada level kedalaman berapa pun.
- **Adjacent Sibling Combinator** (+): memilih elemen yang merupakan sibling langsung (tepat setelah) elemen tertentu. Contoh: `div + p` memilih `<p>` yang tepat berada setelah `<div>`.
- **General Sibling Combinator** (~): memilih elemen yang merupakan sibling yang berada setelah elemen tertentu (tidak harus langsung). Contoh: `div ~p` memilih semua `<p>` yang berada setelah `<div>` dalam parent yang sama.

2.3.3 Selector Chain dan Matching

Selector yang kompleks dapat dibentuk dengan menggabungkan beberapa selector dan combinator dalam sebuah *chain*. Contoh: `div > .box p` berarti memilih elemen `<p>` yang merupakan descendant dari elemen dengan class `box`, yang merupakan direct child dari `<div>`.

Proses *matching* selector dilakukan dengan memeriksa apakah sebuah node DOM memenuhi pola selector yang diberikan. Untuk selector chain, matching dilakukan dari kanan ke kiri, memeriksa hubungan struktural antar elemen sesuai dengan combinator yang digunakan. Setiap combinator memerlukan pengecekan hubungan yang berbeda: child combinator memeriksa parent langsung, descendant combinator menelusuri semua ancestor, dan sibling combinators memeriksa hubungan sibling dalam parent yang sama.

2.4 Breadth-First Search (BFS)

Breadth-First Search (BFS) adalah algoritma traversal graf dan pohon yang menelusuri node-node secara *level-by-level*, mengunjungi semua node pada kedalaman tertentu sebelum berpindah ke kedalaman berikutnya [3]. Dalam konteks pohon DOM, BFS mengunjungi root terlebih dahulu, kemudian semua children dari root, kemudian grandchildren, dan seterusnya.

2.4.1 Karakteristik BFS

BFS menggunakan struktur data *queue* (FIFO — First In First Out) untuk mengelola urutan kunjungan node. Root node dimasukkan ke dalam queue terlebih dahulu.

Selama queue tidak kosong, node di depan queue dikeluarkan untuk diproses, kemudian semua children dari node tersebut dimasukkan ke dalam queue. Proses ini berulang hingga seluruh node dikunjungi atau target ditemukan.

2.4.2 Pseudocode BFS

Algoritma BFS untuk traversal pohon DOM dengan pencarian berdasarkan CSS selector dapat dilihat pada Algoritma 1.

Algorithm 1 BFS untuk Penelusuran Pohon DOM

```
function BFS(root, selectorChain, limit)
    queue ← Queue dengan elemen root
    visited ← 0
    results ← list kosong
    log ← list kosong
    start ← waktu sekarang
    while queue tidak kosong do
        node ← queue.DEQUEUE
        visited ← visited + 1
        if MATCHCHAIN(node, selectorChain) then
            APPEND(log, (visited, node, "match"))
            APPEND(results, node)
            if limit > 0 dan |results| ≥ limit then
                break
            end if
        else
            APPEND(log, (visited, node, "visit"))
        end if
        for setiap child dalam node.Children do
            queue.ENQUEUE(child)
        end for
    end while
    elapsed ← waktu sejak start
    return {results, log, visited, elapsed, "bfs"}
end function
```

2.4.3 Analisis Kompleksitas

Kompleksitas waktu BFS adalah $O(N)$, di mana N adalah jumlah total node dalam pohon, karena setiap node dikunjungi tepat satu kali. Kompleksitas ruang adalah $O(W)$, di mana W adalah lebar maksimum pohon (jumlah maksimum node pada satu level), karena pada saat tertentu queue harus menyimpan semua node pada level yang sedang diproses.

BFS memiliki karakteristik menemukan elemen pada level atas pohon terlebih dahulu. Ini menguntungkan ketika elemen target kemungkinan besar berada di level

yang dangkal. Namun, BFS cenderung boros memori pada pohon yang lebar karena harus menyimpan banyak node dalam queue secara bersamaan.

2.5 Depth-First Search (DFS)

Depth-First Search (DFS) adalah algoritma traversal graf dan pohon yang menelusuri node-node secara mendalam, mengunjungi sejauh mungkin sepanjang setiap cabang sebelum melakukan *backtrack* [3]. Dalam konteks pohon DOM, DFS menelusuri satu subtree secara lengkap sebelum berpindah ke subtree berikutnya.

2.5.1 Karakteristik DFS

DFS dapat diimplementasikan secara iteratif menggunakan *stack* (LIFO — Last In First Out) atau secara rekursif. Dalam implementasi iteratif, root node dimasukkan ke dalam stack terlebih dahulu. Selama stack tidak kosong, node di atas stack dikeluarkan untuk diproses, kemudian children dari node tersebut dimasukkan ke dalam stack dalam urutan terbalik sehingga children paling kiri diproses terlebih dahulu.

2.5.2 Pseudocode DFS

Algoritma DFS untuk traversal pohon DOM dengan pencarian berdasarkan CSS selector dapat dilihat pada Algoritma 2.

2.5.3 Analisis Kompleksitas

Kompleksitas waktu DFS adalah $O(N)$, di mana N adalah jumlah total node dalam pohon, sama seperti BFS. Namun, kompleksitas ruang DFS adalah $O(H)$, di mana H adalah tinggi maksimum pohon (kedalaman terdalam dari leaf node), karena stack hanya menyimpan node-node pada satu path dari root ke leaf yang sedang dijelajahi.

DFS lebih hemat memori pada pohon yang lebar karena tidak perlu menyimpan semua node pada satu level. DFS juga menemukan elemen pada kedalaman tertentu lebih cepat daripada BFS. Namun, DFS tidak menjamin menemukan elemen pada level yang dangkal terlebih dahulu, yang dapat menjadi kelemahan jika target kemungkinan besar berada di level atas pohon.

2.6 Perbandingan BFS dan DFS

BFS dan DFS merupakan dua algoritma traversal fundamental yang memiliki karakteristik berbeda dalam hal urutan kunjungan node, penggunaan memori, dan kesesuaian dengan berbagai skenario pencarian. Tabel 2.1 menyajikan perbandingan komprehensif antara kedua algoritma.

Algorithm 2 DFS untuk Penelusuran Pohon DOM

```
function DFS(root, selectorChain, limit)
    stack ← Stack dengan elemen root
    visited ← 0
    results ← list kosong
    log ← list kosong
    start ← waktu sekarang
    while stack tidak kosong do
        node ← stack.Pop
        visited ← visited + 1
        if MatchChain(node, selectorChain) then
            Append(log, (visited, node, "match"))
            Append(results, node)
            if limit > 0 dan |results| ≥ limit then
                break
            end if
        else
            Append(log, (visited, node, "visit"))
        end if
        for i ← |node.Children| − 1 downto 0 do
            stack.Push(node.Children[i])
        end for
    end while
    elapsed ← waktu sejak start
    return {results, log, visited, elapsed, "dfs"}
end function
```

Tabel 2.1: Perbandingan Algoritma BFS dan DFS

Aspek	**BFS**	**DFS**
Struktur Data	Queue (FIFO)	Stack (LIFO)
Urutan Kunjungan	Level-by-level	Depth-first (subtree lengkap dulu)
Kompleksitas Waktu	$O(N)$	$O(N)$
Kompleksitas Ruang	$O(W)$ — lebar maksimum pohon	$O(H)$ — tinggi maksimum pohon
Menemukan Top-Level Duluan	Ya	Tidak
Menemukan Deep Element Duluan	Tidak	Ya
Cocok untuk Pohon Lebar	Boros memori	Hemat memori
Cocok untuk Pohon Dalam	Hemat memori	Boros memori (jika sangat dalam)

2.6.1 Pemilihan Algoritma yang Tepat

Pemilihan antara BFS dan DFS bergantung pada karakteristik pohon DOM dan sifat pencarian yang dilakukan:

- **Gunakan BFS** ketika:
  - Elemen target kemungkinan besar berada di level dangkal pohon
  - Pohon memiliki struktur yang dalam (tidak terlalu lebar)
  - Perlu menemukan elemen terdekat dari root
  - Memori tidak menjadi kendala utama
- **Gunakan DFS** ketika:
  - Elemen target kemungkinan besar berada di kedalaman tertentu
  - Pohon memiliki struktur yang lebar (banyak children per node)
  - Perlu menelusuri satu subtree lengkap sebelum berpindah
  - Penggunaan memori perlu diminimalkan

Dalam konteks penelusuran CSS Selector pada pohon DOM, kedua algoritma dapat digunakan bergantung pada pola dokumen HTML yang umum dijumpai. BFS menguntungkan untuk selector yang menargetkan elemen umum seperti `<p>` atau `<div>` yang biasanya tersebar di berbagai level. DFS menguntungkan untuk selector dengan combinator descendant (`div p`) karena dapat menelusuri satu cabang pohon secara lengkap.

2.7 Web Scraping

Web scraping adalah teknik ekstraksi data dari situs web dengan mengambil konten halaman web dan mengubahnya menjadi format yang dapat diproses secara terstruktur [8]. Dalam konteks program Berry-Manuka, web scraping digunakan untuk mengambil dokumen HTML dari URL yang diberikan pengguna sebelum dokumen tersebut diparse menjadi pohon DOM.

2.7.1 HTTP Client dan Request Handling

Proses web scraping dimulai dengan membuat HTTP request ke URL target menggunakan *HTTP client*. Request yang umum digunakan untuk mengambil halaman web adalah HTTP GET request. HTTP client mengirimkan request ke peladen web dan menerima response yang berisi status code, headers, dan body (konten halaman).

Headers dalam HTTP response memberikan informasi penting seperti:

- `Content-Type`: menunjukkan tipe konten (harus `text/html` untuk dokumen HTML)
- `Content-Length`: ukuran konten dalam bytes
- `Status Code`: hasil permintaan (200 OK untuk sukses)

2.7.2 Validasi dan Error Handling

Beberapa aspek penting dalam implementasi web scraping yang robust:

- **URL Validation**: memastikan URL valid dengan skema yang benar (`http://` atau `https://`) dan format yang sesuai.
- **Timeout**: membatasi waktu tunggu response untuk mencegah program hang jika peladen tidak merespons. Timeout yang umum digunakan adalah 10-30 detik.
- **Size Limit**: membatasi ukuran konten yang diunduh untuk mencegah penggunaan memori yang berlebihan. Batas yang umum adalah 5-10 MB.
- **Error Handling**: menangani berbagai skenario error seperti connection timeout, DNS resolution failure, HTTP error codes (404, 500, dll), dan invalid content type.

2.7.3 Security Considerations

Web scraping perlu memperhatikan aspek keamanan dan etika:

- **Robots.txt**: menghormati file `robots.txt` yang menentukan bagian situs mana yang boleh atau tidak boleh di-scrape.

- **Rate Limiting**: tidak membuat request terlalu sering ke satu situs untuk menghindari beban berlebih pada peladen.
- **Static vs Dynamic Content**: web scraping menggunakan HTTP client standar hanya dapat mengambil konten *static*. Konten yang dirender menggunakan JavaScript (*dynamic content*) memerlukan tools khusus seperti *headless browser*.

Dalam implementasi Berry-Manuka, web scraping dilakukan menggunakan paket `net/http` dari bahasa Go yang menyediakan HTTP client dengan dukungan *timeout*, *redirect handling*, dan validasi *content-type*.

2.8 Lowest Common Ancestor dan Binary Lifting

Lowest Common Ancestor (LCA) dari dua node dalam pohon adalah node terdalam yang merupakan ancestor dari kedua node tersebut [4]. Dalam konteks pohon DOM, LCA merepresentasikan elemen kontainer terdekat yang mengandung kedua elemen yang dipilih.

2.8.1 Definisi LCA

Diberikan pohon T dengan root r dan dua node u dan v dalam pohon tersebut, LCA dari u dan v (dinotasikan $\text{LCA}(u, v)$) adalah node w dengan kedalaman maksimum sedemikian sehingga w adalah ancestor dari u dan w juga adalah ancestor dari v.

2.8.2 Pendekatan Naive

Pendekatan sederhana untuk mencari LCA adalah dengan menelusuri parent dari kedua node secara bersamaan hingga ditemukan node yang sama. Algoritma ini:

1. Samakan kedalaman kedua node dengan menggerakkan node yang lebih dalam ke atas (menuju parent) hingga kedalaman sama.
2. Gerakkan kedua node ke atas bersamaan hingga keduanya bertemu di node yang sama. Node pertemuan tersebut adalah LCA.

Kompleksitas pendekatan naive adalah $O(N)$ per query dalam kasus terburuk, di mana N adalah jumlah node dalam pohon.

2.8.3 Binary Lifting

Binary Lifting adalah teknik optimasi yang memungkinkan query LCA dalam kompleksitas $O(\log N)$ dengan preprocessing $O(N \log N)$ [9]. Teknik ini menggunakan tabel *sparse table* untuk menyimpan ancestor 2^k dari setiap node.

Preprocessing

Didefinisikan up$[v][k]$ sebagai ancestor ke-2^k dari node v:

- up$[v][0]$ = parent(v) (ancestor $2^0 = 1$)
- up$[v][k]$ = up$[$up$[v][k-1]][k-1]$ untuk $k > 0$ (ancestor 2^k adalah ancestor 2^{k-1} dari ancestor 2^{k-1})

Tabel diisi menggunakan BFS atau DFS dari root. Nilai MAXLOG dipilih sedemikian sehingga $2^{\text{MAXLOG}} > N$. Untuk pohon dengan hingga sekitar 1 juta node, MAXLOG = 20 sudah mencukupi karena $2^{20} = 1.048.576$.

Query LCA

Algoritma query LCA dengan Binary Lifting:

1. **Samakan kedalaman**: jika depth$[u]$ > depth$[v]$, tukar u dan v. Kemudian naikkan u menggunakan binary lifting hingga depth$[u]$ = depth$[v]$.
2. **Binary lift bersama**: jika $u = v$ setelah langkah 1, return u. Jika tidak, naikkan u dan v bersamaan dari $k = \text{MAXLOG} - 1$ hingga 0, tetapi hanya jika up$[u][k] \neq$ up$[v][k]$. Ini memastikan kita tidak melewati LCA.
3. **Return parent**: LCA adalah parent dari u (atau v), yaitu up$[u][0]$.

Teknik ini juga memungkinkan rekonstruksi jalur dari suatu node ke root dengan kompleksitas $O(\log N)$ per node dalam jalur.

2.9 Konkurensi dan Goroutine

Concurrency adalah kemampuan suatu program untuk menjalankan beberapa tugas secara bersamaan atau tumpang tindih dalam waktu [10]. Concurrency berbeda dengan *parallelism*: concurrency berkaitan dengan struktur program yang memungkinkan beberapa tugas berjalan secara bersamaan, sedangkan parallelism berkaitan dengan eksekusi fisik beberapa tugas secara simultan pada beberapa prosesor.

2.9.1 Goroutine

Goroutine adalah unit eksekusi ringan (*lightweight*) yang disediakan oleh bahasa pemrograman Go untuk mendukung concurrency. Berbeda dengan *thread* pada umumnya yang dikelola oleh sistem operasi, goroutine dikelola oleh *runtime* Go sehingga lebih efisien dalam penggunaan memori (biasanya hanya beberapa KB) dan lebih cepat dalam pembuatan serta penghancurannya.

Goroutine dibuat dengan menambahkan kata kunci `go` sebelum pemanggilan fungsi:

```
go namaFungsi(argumen)
```

Program utama (`main`) dan semua goroutine yang aktif berjalan secara concurrent, memungkinkan pemanfaatan multi-core processor untuk parallelism.

2.9.2 Sinkronisasi dengan WaitGroup

WaitGroup dari paket `sync` adalah mekanisme sinkronisasi yang memungkinkan sebuah goroutine menunggu selesainya sejumlah goroutine lain. WaitGroup memiliki tiga metode utama:

- `Add(n)`: menambah counter sebesar n
- `Done()`: mengurangi counter (dipanggil oleh goroutine yang selesai)
- `Wait()`: blok sampai counter mencapai nol

2.9.3 Thread Safety dengan Mutex

Mutex (*mutual exclusion*) dari paket `sync` adalah mekanisme penguncian yang memastikan hanya satu goroutine yang mengakses sumber daya bersama pada satu waktu. Ini mencegah *race condition* ketika beberapa goroutine mengakses dan memodifikasi data bersama secara bersamaan.

Mutex memiliki dua metode utama:

- `Lock()`: mengakuisisi lock. Jika lock sedang dipegang goroutine lain, goroutine ini akan blok sampai lock tersedia.
- `Unlock()`: melepaskan lock, memungkinkan goroutine lain mengakuisisinya.

2.9.4 Parallel BFS dan DFS

Parallel BFS dapat diimplementasikan dengan memproses semua node pada level yang sama secara paralel menggunakan goroutine. Setiap node diproses oleh goroutine terpisah, dan hasilnya dikumpulkan menggunakan mutex untuk melindungi akses ke struktur data bersama.

Parallel DFS dapat diimplementasikan dengan memproses setiap subtree langsung dari root secara paralel. Setiap child dari root diproses oleh goroutine terpisah yang menjalankan DFS sequential pada subtree tersebut.

Keuntungan parallel traversal tergantung pada ukuran pohon dan struktur hardware. Untuk pohon yang cukup besar, overhead pembuatan goroutine dapat diimbangi oleh gain dari eksekusi paralel. Namun, untuk pohon kecil, sequential traversal mungkin lebih cepat karena tidak ada overhead sinkronisasi.

2.10 Containerization dan Docker

Containerization adalah teknik virtualisasi tingkat sistem operasi yang memungkinkan aplikasi dan dependensinya dikemas dalam sebuah *container* yang terisolasi [11]. Container menyediakan lingkungan runtime yang konsisten dan portable, memastikan aplikasi berjalan sama di berbagai environment (development, testing, production).

2.10.1 Konsep Container

Container berbeda dengan *virtual machine* (VM) tradisional. VM mengemulasikan seluruh sistem operasi termasuk kernel, sedangkan container berbagi kernel sistem operasi host dan hanya mengisolasi proses, filesystem, dan network namespace. Ini membuat container jauh lebih ringan dan cepat dibandingkan VM.

Keuntungan menggunakan container:

- **Portability**: aplikasi dalam container dapat berjalan di environment mana pun yang mendukung container runtime
- **Isolation**: setiap container memiliki filesystem, proses, dan network yang terpisah, mencegah konflik antar aplikasi
- **Efficiency**: container menggunakan sumber daya lebih efisien daripada VM karena berbagi kernel OS
- **Scalability**: container dapat dengan mudah direplikasi dan di-*orchestrate* menggunakan tools seperti Docker Compose atau Kubernetes

2.10.2 Docker

Docker adalah platform containerization yang paling populer, menyediakan tools untuk membangun, menjalankan, dan mengelola container. Komponen utama Docker meliputi:

- **Docker Engine**: runtime container yang menjalankan container
- **Dockerfile**: file konfigurasi yang mendefinisikan cara membangun *image* container
- **Docker Image**: template read-only untuk membuat container
- **Docker Compose**: tool untuk mendefinisikan dan menjalankan multi-container applications

2.10.3 Dockerfile dan Multi-Stage Builds

Dockerfile mendefinisikan instruksi untuk membangun image container. *Multi-stage builds* adalah teknik yang menggunakan beberapa `FROM` statements dalam satu Dockerfile, memungkinkan pemisahan antara environment build dan runtime.

Keuntungan multi-stage builds:

- Mengurangi ukuran final image dengan hanya menyertakan artifacts yang diperlukan (binary executable), tidak termasuk tools build
- Meningkatkan keamanan dengan tidak menyertakan compiler atau tools development dalam image production
- Mempercepat deployment karena image yang lebih kecil lebih cepat di-push dan di-pull

2.10.4 Docker Compose

Docker Compose adalah tool untuk mendefinisikan dan menjalankan aplikasi multi-container. Dengan menggunakan file `docker-compose.yml`, seluruh stack aplikasi (frontend, backend, database, dll) dapat didefinisikan secara deklaratif dan dijalankan dengan satu perintah.

Docker Compose mengelola:

- Pembuatan dan konfigurasi container
- Network antara container
- Volume untuk persistent storage
- Dependency dan urutan startup antar services
- Environment variables dan port mapping

Dalam konteks program Berry-Manuka, Docker Compose digunakan untuk mengorkestrasi dua container: backend (Go) dan frontend (Next.js), memastikan kedua service dapat berkomunikasi dan dijalankan bersama dengan konfigurasi yang konsisten.

Bab 3

Aplikasi BFS dan DFS

3.1 Representasi DOM Tree

Struktur DOM direpresentasikan sebagai tree, dengan setiap node memiliki atribut sebagai berikut:

Atribut	Deskripsi
Tag	Nama elemen HTML
Attribute	Atribut pada tag
Children	Node anak
Parent	Node induk

Struktur ini memungkinkan traversal dilakukan secara sistematis menggunakan BFS maupun DFS.

3.2 HTML Parsing

HTML yang diperoleh dari input akan diproses menjadi struktur DOM Tree melalui proses parsing.

Alur Parsing

HTML Input → Tokenisasi → Tree Construction → DOM Tree

Tahapan:

1. Membaca HTML mentah
2. Mengidentifikasi tag pembuka dan penutup
3. Menyusun node dalam bentuk tree

3.3 Implementasi CSS Selector

CSS Selector digunakan untuk menentukan elemen target dalam traversal.

Tipe	Contoh	Fungsi
Tag	p	Semua elemen p
Class	.box	Elemen dengan class tertentu
ID	#header	Elemen dengan id tertentu

Combinator yang digunakan:

- Descendant ( )
- Child (>)
- Adjacent Sibling (+)
- General Sibling ( )

3.4 Implementasi Algoritma BFS dan DFS

3.4.1 Implementasi Algoritma BFS

Implementasi algoritma *Breadth First Search* (BFS) pada aplikasi ini digunakan untuk menelusuri pohon DOM secara melebar, yaitu mengunjungi seluruh node pada satu level terlebih dahulu sebelum melanjutkan ke level berikutnya. Pendekatan ini sesuai untuk kasus pencarian elemen yang cenderung berada lebih dekat ke akar pohon.

Kode program BFS diambil langsung dari file implementasi pada repository, yaitu:

Lokasi File Implementasi BFS

`backend/internal/traversal/bfs.go`

```go
package traversal

import (
    "time"
    "tubes2/model"
    "tubes2/selector"
)

func BFS(root *model.DOMNode, chain model.SelectorChain, limit int) model.TraversalResult {
    start := time.Now()
    result := model.TraversalResult{
        MatchedNodes: make([]*model.DOMNode, 0),
        Log:          make([]model.LogEntry, 0),
        Algorithm:    "bfs",
    }

    if root == nil {
```

```go
        result.ExecutionMs = float64(time.Since(start).Microseconds()) / 1000.0
        return result
    }

    model.RebuildParentPointers(root)
    result.MaxDepth = model.MaxDepth(root)

    queue := []*model.DOMNode{root}
    step := 0

    for len(queue) > 0 {
        node := queue[0]
        queue = queue[1:]
        step++
        result.VisitedCount++

        action := "visit"
        if selector.MatchChain(node, chain) {
            action = "match"
            result.MatchedNodes = append(result.MatchedNodes, node)
        }

        result.Log = append(result.Log, model.LogEntry{
            Step:   step,
            NodeID: node.ID,
            Tag:    node.Tag,
            Action: action,
            Depth:  node.Depth,
        })

        if limit > 0 && len(result.MatchedNodes) >= limit {
            break
        }

        queue = append(queue, node.Children...)
    }

    result.ExecutionMs = float64(time.Since(start).Microseconds()) / 1000.0
    return result
}
```

Listing 3.1: Implementasi algoritma BFS pada file `bfs.go`

Secara umum, alur kerja BFS pada pohon DOM dalam program ini dapat diringkas pada Tabel 3.1.

Tabel 3.1: Alur kerja BFS pada DOM Tree

Langkah	Penjelasan
1	Node akar DOM dimasukkan ke dalam *queue*.
2	Node paling depan diambil dari *queue*.
3	Node diperiksa apakah sesuai dengan CSS selector yang diberikan.
4	Jika sesuai, node dicatat sebagai hasil pencarian.
5	Seluruh child dari node tersebut dimasukkan ke *queue*.
6	Proses diulangi sampai *queue* kosong atau jumlah hasil yang diminta telah terpenuhi.

Karakteristik utama BFS pada implementasi ini ditunjukkan pada ringkasan berikut.

Aspek	Keterangan
Struktur data	Menggunakan *queue* (FIFO).
Pola traversal	Menelusuri pohon per level, dari atas ke bawah.
Kelebihan	Baik untuk menemukan elemen yang berada dekat root lebih awal.
Kekurangan	Dapat menggunakan memori lebih besar pada pohon yang sangat lebar.

Dengan demikian, implementasi BFS pada aplikasi ini menekankan traversal level-order sehingga urutan kunjungan node dapat divisualisasikan dengan jelas pada traversal log dan highlight jalur pencarian.

3.4.2 Implementasi Algoritma DFS

Implementasi algoritma *Depth First Search* (DFS) pada aplikasi ini digunakan untuk menelusuri pohon DOM secara mendalam, yaitu mengikuti satu cabang hingga node terdalam terlebih dahulu sebelum melakukan *backtracking* ke cabang lain. Pendekatan ini sesuai untuk eksplorasi struktur DOM yang dalam.

Kode program DFS diambil langsung dari file implementasi pada repository, yaitu:

Lokasi File Implementasi DFS

`backend/internal/traversal/dfs.go`

```
package traversal

import (
    "time"

    "tubes2/model"
    "tubes2/selector"
)
```

```go

func DFS(root *model.DOMNode, chain model.SelectorChain, limit int) model.TraversalResult {
    start := time.Now()
    result := model.TraversalResult{
        MatchedNodes: make([]*model.DOMNode, 0),
        Log:          make([]model.LogEntry, 0),
        Algorithm:    "dfs",
    }

    if root == nil {
        result.ExecutionMs = float64(time.Since(start).Microseconds()) / 1000.0
        return result
    }

    model.RebuildParentPointers(root)
    result.MaxDepth = model.MaxDepth(root)

    stack := []*model.DOMNode{root}
    step := 0

    for len(stack) > 0 {
        lastIdx := len(stack) - 1
        node := stack[lastIdx]
        stack = stack[:lastIdx]
        step++
        result.VisitedCount++

        action := "visit"
        if selector.MatchChain(node, chain) {
            action = "match"
            result.MatchedNodes = append(result.MatchedNodes, node)
        }

        result.Log = append(result.Log, model.LogEntry{
            Step:   step,
            NodeID: node.ID,
            Tag:    node.Tag,
            Action: action,
            Depth:  node.Depth,
        })

        if limit > 0 && len(result.MatchedNodes) >= limit {
            break
        }

        for i := len(node.Children) - 1; i >= 0; i-- {
```

```
            stack = append(stack, node.Children[i])
        }
    }

    result.ExecutionMs = float64(time.Since(start).
        Microseconds()) / 1000.0
    return result
}
```

Listing 3.2: Implementasi algoritma DFS pada file `dfs.go`

Secara umum, alur kerja DFS pada pohon DOM dalam program ini dapat diringkas pada Tabel 3.2.

Tabel 3.2: Alur kerja DFS pada DOM Tree

Langkah	**Penjelasan**
1	Traversal dimulai dari node akar DOM.
2	Node saat ini diperiksa apakah sesuai dengan CSS selector.
3	Jika sesuai, node dicatat sebagai hasil pencarian.
4	Algoritma melanjutkan penelusuran ke child node secara mendalam.
5	Jika suatu cabang selesai ditelusuri, algoritma kembali ke parent untuk melanjutkan ke cabang lain.
6	Proses berhenti ketika seluruh node telah ditelusuri atau jumlah hasil yang diminta telah terpenuhi.

Karakteristik utama DFS pada implementasi ini ditunjukkan pada ringkasan berikut.

Aspek	**Keterangan**
Struktur data	Menggunakan *stack* atau rekursi.
Pola traversal	Menelusuri satu cabang hingga dalam sebelum pindah ke cabang lain.
Kelebihan	Cocok untuk pohon yang dalam dan relatif hemat memori pada beberapa kasus.
Kekurangan	Tidak selalu menemukan node yang dekat root lebih dahulu.

Dengan demikian, implementasi DFS pada aplikasi ini menekankan traversal depth-first, sehingga jalur pencarian terlihat lebih fokus pada satu cabang sebelum berpindah ke cabang lainnya. Hal ini juga menghasilkan pola traversal log yang berbeda dibandingkan BFS.

3.4.3 Perbandingan BFS dan DFS

Aspek	BFS	DFS
Traversal	Level	Depth
Struktur Data	Queue	Stack / Rekursi
Kecepatan	Lebih cepat untuk target dekat	Lebih cepat untuk struktur dalam
Memory	Lebih besar	Lebih kecil

3.4.4 Analisis Kompleksitas

- BFS: O(n)
- DFS: O(n)

Dengan:

- n = jumlah node pada DOM

Performa juga dipengaruhi oleh:

- Kompleksitas selector
- Struktur pohon DOM

3.5 Implementasi Binary Lifting untuk LCA pada DOM

Salah satu bonus yang diimplementasikan pada aplikasi ini adalah pencarian *Lowest Common Ancestor* (LCA) menggunakan teknik *Binary Lifting*. Dalam konteks pohon DOM, fitur ini digunakan untuk menentukan elemen HTML terdekat yang menjadi leluhur bersama dari dua node tertentu. Implementasi ini disimpan pada file `backend/lca/binary_lifting.go`.

Lokasi File Implementasi Binary Lifting

`backend/lca/binary_lifting.go`

```go
package lca

import "tubes2/model"

const maxLog = 20

type LCAProcessor struct {
    Up    [][]int
    Depth []int
    Nodes []*model.DOMNode
    N     int
}

```

```go
func NewLCAProcessor(root *model.DOMNode) *LCAProcessor {
    model.RebuildParentPointers(root)

    totalNodes := countNodes(root)
    up := make([][]int, totalNodes)
    for i := range up {
        up[i] = make([]int, maxLog)
        for j := range up[i] {
            up[i][j] = -1
        }
    }
    depth := make([]int, totalNodes)
    nodes := make([]*model.DOMNode, totalNodes)

    fillTables(root, up, depth, nodes)

    for k := 1; k < maxLog; k++ {
        for v := 0; v < totalNodes; v++ {
            if up[v][k-1] != -1 {
                up[v][k] = up[up[v][k-1]][k-1]
            }
        }
    }

    return &LCAProcessor{
        Up:    up,
        Depth: depth,
        Nodes: nodes,
        N:     totalNodes,
    }
}

func countNodes(node *model.DOMNode) int {
    if node == nil {
        return 0
    }
    count := 1
    for _, child := range node.Children {
        count += countNodes(child)
    }
    return count
}

func fillTables(node *model.DOMNode, up [][]int, depth []int,
    nodes []*model.DOMNode) {
    if node == nil {
        return
    }
    depth[node.ID] = node.Depth
```

```
    nodes[node.ID] = node
    if node.ParentID == -1 {
        up[node.ID][0] = -1
    } else {
        up[node.ID][0] = node.ParentID
    }
    for _, child := range node.Children {
        fillTables(child, up, depth, nodes)
    }
}

func (lca *LCAProcessor) Query(u, v int) int {
    if lca.Depth[u] < lca.Depth[v] {
        u, v = v, u
    }

    diff := lca.Depth[u] - lca.Depth[v]
    for k := 0; k < maxLog; k++ {
        if (diff>>k)&1 == 1 {
            u = lca.Up[u][k]
        }
    }

    if u == v {
        return u
    }

    for k := maxLog - 1; k >= 0; k-- {
        if lca.Up[u][k] != lca.Up[v][k] {
            u = lca.Up[u][k]
            v = lca.Up[v][k]
        }
    }

    return lca.Up[u][0]
}

func (lca *LCAProcessor) GetPath(nodeID int) []int {
    var path []int
    for nodeID != -1 {
        path = append(path, nodeID)
        nodeID = lca.Up[nodeID][0]
    }
    return path
}

func (lca *LCAProcessor) FindLCA(idA, idB int) *model.DOMNode
    {
    result := lca.Query(idA, idB)
```

```
    return lca.Nodes[result]
}
```

Listing 3.3: Implementasi Binary Lifting pada file `binary_lifting.go`

Secara umum, implementasi ini dibangun di atas empat komponen utama yang dirangkum pada Tabel 3.3.

Tabel 3.3: Komponen utama implementasi Binary Lifting

Komponen	**Peran**
`LCAProcessor`	Menyimpan seluruh tabel dan informasi yang diperlukan untuk query LCA.
`NewLCAProcessor`	Melakukan preprocessing terhadap pohon DOM dan membangun tabel ancestor.
`Query`	Menghitung ID node LCA dari dua node yang diberikan.
`GetPath` dan `FindLCA`	Fungsi tambahan untuk mengambil jalur node ke root dan mengembalikan node DOM hasil LCA.

Struktur Data yang Digunakan

Struktur utama yang digunakan adalah `LCAProcessor`. Struktur ini menyimpan informasi yang diperlukan agar pencarian LCA dapat dilakukan dengan cepat.

Field	**Fungsi**
`Up [][]int`	Tabel ancestor, dengan `Up[v][k]` menyatakan leluhur ke-2^k dari node `v`.
`Depth []int`	Menyimpan kedalaman setiap node pada pohon DOM.
`Nodes []*model.DOMNode`	Menyimpan referensi node DOM berdasarkan ID, sehingga hasil query dapat langsung dipetakan kembali ke node asli.
`N int`	Menyimpan jumlah total node.

Konstanta `maxLog = 20` digunakan sebagai batas jumlah level ancestor yang disimpan. Nilai ini berarti implementasi dapat menangani kedalaman hingga sekitar 2^{20} langkah ancestor, yang sudah sangat cukup untuk ukuran DOM pada aplikasi ini.

Tahap Preprocessing

Tahap preprocessing dilakukan di fungsi `NewLCAProcessor`. Tahap ini bertujuan membangun tabel ancestor sebelum query dijalankan. Alurnya dirangkum pada Tabel 3.4.

Tabel 3.4: Alur preprocessing Binary Lifting

Langkah	Penjelasan
1	Melakukan `RebuildParentPointers(root)` agar setiap node memiliki informasi parent yang konsisten.
2	Menghitung jumlah total node dengan fungsi `countNodes(root)`.
3	Mengalokasikan tabel `up`, array `depth`, dan array `nodes`.
4	Menginisialisasi seluruh nilai `up[v][k]` dengan `-1` sebagai penanda bahwa ancestor belum ada.
5	Mengisi `depth`, `nodes`, dan `up[v][0]` melalui fungsi `fillTables`.
6	Mengisi tabel ancestor tingkat lebih tinggi dengan rumus `up[v][k] = up[up[v][k-1]][k-1]`.

Bagian preprocessing inilah yang menjadi inti dari teknik Binary Lifting. Setelah tabel ancestor selesai dibangun, query LCA tidak perlu lagi menelusuri pohon dari awal.

Intuisi Tabel `Up`

Jika `Up[v][0]` adalah parent langsung dari node `v`, maka:

- `Up[v][1]` adalah leluhur 2 langkah di atas `v`
- `Up[v][2]` adalah leluhur 4 langkah di atas `v`
- `Up[v][3]` adalah leluhur 8 langkah di atas `v`

Dengan demikian, perpindahan ke ancestor dapat dilakukan dalam lompatan pangkat dua.

Fungsi Pendukung

Terdapat dua fungsi pendukung utama sebelum query dilakukan.

1. `countNodes(node)`
   Fungsi ini menghitung jumlah seluruh node pada DOM Tree secara rekursif. Nilai ini digunakan untuk menentukan ukuran array yang akan dialokasikan.

2. `fillTables(node, up, depth, nodes)`
   Fungsi ini mengisi tiga informasi dasar:

   - `depth[node.ID]` dengan kedalaman node,
   - `nodes[node.ID]` dengan referensi node DOM,
   - `up[node.ID][0]` dengan parent langsung node tersebut.

Karena setiap node DOM pada program ini memiliki `ID`, `Depth`, dan `ParentID`, maka proses pengisian tabel dapat dilakukan secara langsung tanpa perlu pemetaan tambahan.

Proses Query LCA

Pencarian LCA dilakukan melalui fungsi `Query(u, v)`. Fungsi ini menerima dua ID node, kemudian mengembalikan ID dari leluhur bersama terendah.

Secara garis besar, proses query terdiri atas dua tahap yang ditunjukkan pada Tabel 3.5.

Tabel 3.5: Alur query LCA dengan Binary Lifting

Tahap	Penjelasan
1	Jika kedalaman kedua node berbeda, node yang lebih dalam diangkat terlebih dahulu sampai kedalamannya sama.
2	Jika kedua node belum sama, keduanya diangkat bersama-sama dari lompatan terbesar ke terkecil sampai parent langsung keduanya sama.

Penjelasannya adalah sebagai berikut:

- Mula-mula, fungsi memastikan bahwa `u` berada pada kedalaman yang sama atau lebih dalam daripada `v`.
- Selisih kedalaman dihitung dalam variabel `diff`.
- Dengan memanfaatkan representasi biner dari `diff`, node `u` diangkat menggunakan tabel `Up`.
- Jika setelah itu `u == v`, berarti salah satu node memang ancestor dari node lainnya.
- Jika belum sama, maka kedua node diangkat serentak dari level tertinggi ke terendah.
- Ketika keduanya hampir bertemu, hasil akhirnya adalah parent langsung dari salah satunya, yaitu `lca.Up[u][0]`.

Keunggulan Query Ini

Setelah preprocessing selesai, pencarian LCA tidak lagi memerlukan traversal penuh pada pohon DOM. Query dapat diselesaikan dalam kompleksitas waktu $\mathcal{O}(\log N)$.

Fungsi Tambahan

Selain fungsi query utama, terdapat dua fungsi tambahan yang berguna untuk integrasi dengan fitur visualisasi.

1. `GetPath(nodeID)`
   Fungsi ini menghasilkan jalur dari suatu node menuju root dengan terus mengikuti parent langsung, yaitu `Up[nodeID][0]`. Fungsi ini sangat berguna untuk menampilkan path pada visualisasi frontend.

2. `FindLCA(idA, idB)`
   Fungsi ini merupakan pembungkus dari `Query`. Setelah mendapatkan ID LCA, fungsi ini langsung mengembalikan pointer ke `DOMNode` melalui array `Nodes`. Dengan demikian, hasil query dapat langsung digunakan pada sisi aplikasi.

Analisis Implementasi

Implementasi Binary Lifting ini cukup sesuai untuk kasus pohon DOM karena struktur DOM bersifat hierarkis dan setiap node memiliki parent tunggal. Ringkasan analisisnya dapat dilihat pada Tabel 3.6.

Tabel 3.6: Analisis implementasi Binary Lifting

Aspek	**Keterangan**
Struktur yang diproses	Pohon DOM hasil parsing HTML.
Preprocessing	Mengisi parent langsung, depth, dan tabel ancestor.
Kompleksitas preprocessing	$\mathcal{O}(N \log N)$
Kompleksitas query LCA	$\mathcal{O}(\log N)$
Kelebihan	Sangat efisien untuk banyak query LCA pada pohon yang sama.
Keterbatasan	Memerlukan memori tambahan untuk menyimpan tabel `Up`.

Secara keseluruhan, implementasi ini sudah terstruktur dengan baik. Fungsi `NewLCAProcessor` bertugas membangun seluruh data bantu, sedangkan `Query` menangani pencarian LCA secara efisien. Tambahan fungsi `GetPath` dan `FindLCA` juga membuat modul ini lebih mudah diintegrasikan dengan fitur visualisasi jalur dan penandaan node pada aplikasi.

Bab 4

Implementasi

4.1 Gambaran Umum Sistem

Aplikasi yang dikembangkan merupakan sistem berbasis web yang bertujuan untuk melakukan pencarian elemen HTML pada struktur DOM berdasarkan CSS selector. Sistem ini memanfaatkan algoritma Breadth First Search (BFS) dan Depth First Search (DFS) sebagai metode traversal.

Input dan Output Sistem

Input:

- URL website atau teks HTML
- CSS Selector
- Pilihan algoritma (BFS / DFS)
- Jumlah hasil

Output:

- Elemen hasil pencarian
- Visualisasi DOM
- Jalur traversal
- Waktu eksekusi

4.2 Arsitektur Sistem

Sistem dibangun dengan pendekatan client-server.

Frontend	→	Backend
Input user		Parsing + BFS/DFS
Visualisasi	←	Hasil traversal

- Frontend: menerima input dan menampilkan hasil
- Backend: melakukan parsing HTML dan traversal DOM

Bab 5

Penutup

5.1 Kesimpulan

5.2 Saran

Bab 6

Lampiran

Bibliografi

[1] MDN Web Docs, "Document object model (DOM)," https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model, 2026, accessed: Apr-2026.

[2] ——, "CSS selectors," https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_selectors, 2026, accessed: Apr-2026.

[3] T. Cormen, C. Leiserson, R. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.

[4] CP-Algorithms, "Lowest common ancestor — Binary Lifting," https://cp-algorithms.com/graph/lca_binary_lifting.html, 2026, accessed: Apr-2026.

[5] D. Flanagan, *JavaScript: The Definitive Guide*, 5th ed. O'Reilly Media, 2006.

[6] MDN Web Docs, "HTML: Hypertext markup language," https://developer.mozilla.org/en-US/docs/Web/HTML, 2026, accessed: Apr-2026.

[7] WHATWG, "HTML standard," https://html.spec.whatwg.org/, 2026, accessed: Apr-2026.

[8] W3Schools, "What is web scraping?" https://www.w3schools.com/python/python_scraping.asp, 2026, accessed: Apr-2026.

[9] M. A. Bender and M. Farach-Colton, "The LCA problem revisited," *Proceedings of the 4th Latin American Symposium on Theoretical Informatics*, pp. 88–94, 2000.

[10] A. A. Donovan and B. W. Kernighan, *The Go Programming Language*. Addison-Wesley, 2015.

[11] Docker Inc., "Docker documentation," https://docs.docker.com/, 2026, accessed: Apr-2026.